

UNIVERSIDAD NACIONAL DEL NORDESTE

FACENA

PARADIGMAS Y LENGUAJES DE LA PROGRAMACIÓN

TRABAJO INTEGRADOR

Sistema de Semáforos Inteligentes

y

**Análisis Comparativo de Paradigmas
(Clojure)**

INTEGRANTES (GRUPO 26)

Díaz Bruno Leandro

Lucas Agustín Lindström Suñol

Rita Gonzalez Coene

Aegerter Sebastián Gabriel

William Abdon Alarcon

Fundamentos del Diseño Funcional

El sistema de semáforos se desarrolló aplicando las bases de la programación funcional sobre Common Lisp. La estructura del código se apoya en cuatro conceptos clave:

Uso de Funciones Puras: Las rutinas principales del programa se diseñaron para que sus resultados dependan únicamente de los parámetros que reciben. Al no usar ni modificar variables globales, el comportamiento del control de tráfico se vuelve totalmente predecible y fácil de probar.

Inmutabilidad del Estado: Los cambios de luces (verde, amarillo, rojo y sus intermitencias) no sobrescriben ni destruyen la información en la memoria. Cada transición calcula un estado nuevo a partir de las reglas lógicas, lo que evita que los datos se corrompan durante la ejecución.

Modularidad y Composición de Funciones: El problema se dividió en funciones chicas y específicas (validar secuencias, contar el tiempo y calcular porcentajes). Estas funciones se conectan directamente entre sí, logrando que la salida de un módulo pase como dato de entrada al siguiente sin pasos intermedios.

Uso de Expresiones de Lógica Declarativa: En lugar de usar bucles tradicionales con contadores que van cambiando (estilo *for* o *while* del paradigma imperativo), los ciclos de 216 y 225 segundos se resolvieron mediante la evaluación de expresiones condicionales, dejando que el flujo sea manejado por la propia lógica del lenguaje.

Justificación de la utilización de Local-Time

Durante las tareas de depuración relacionadas con la obtención del tiempo Unix (*Unix Timestamp*), se identificó una forma más adecuada de realizar dicha operación mediante la biblioteca **Local-Time**.

En una primera versión del proyecto, el cálculo del timestamp se realizaba utilizando únicamente funciones estándar de Common Lisp. Sin embargo, la incorporación de Local-Time permitió simplificar significativamente la implementación, obteniendo una solución más clara, compacta y fácil de mantener.

Como resultado, la función encargada de obtener el timestamp fue refactorizada para utilizar las utilidades provistas por esta biblioteca, mejorando la legibilidad del código y reduciendo la complejidad de la implementación.

Integración de la biblioteca Local-Time

Con Quicklisp funcionando correctamente sobre SBCL, se procedió a instalar la biblioteca **Local-Time**, utilizada para el manejo de fechas y horas.

```
* (ql:quickload "local-time")
To load "local-time":
  Load 2 ASDF systems:
    asdf uiop
  Install 1 Quicklisp release:
    local-time
; Fetching #<URL "http://beta.quicklisp.org/archive/local-time/2026-01-01/local-time-20260101-git.tgz">
; 550.30KB
=====
563,503 bytes in 0.42 seconds (1305.30KB/sec)
; Loading "local-time"
[package local-time].....
("local-time")
```

La instalación se completó satisfactoriamente, incorporando la biblioteca al entorno de desarrollo.

Problema durante la instalación de Quicklisp

Durante la carga del archivo quicklisp.lisp se produjo repetidamente el siguiente error:

```
[1]> (load "C:/Users/crist/Downloads/quicklisp.lisp")

*** - Error de Win32 267 (ERROR_DIRECTORY): The directory name is invalid.
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort main loop
```

Luego de analizar el problema, se determinó que la causa estaba relacionada con la implementación de Common Lisp utilizada inicialmente, CLISP. Debido a sus limitaciones y a la falta de compatibilidad con algunas bibliotecas modernas, se decidió migrar a una implementación más actual y ampliamente utilizada: **SBCL (Steel Bank Common Lisp)**.

Bitácora Pormenorizada de Bugs y Depuración

Bug N° 1: Retorno de números decimales (flotantes) en divisiones temporales

Contexto: Al desarrollar la función `ciclos-por-tiempo` (Requerimiento 5).

Problema encontrado: Al ejecutar la división para calcular cuántos ciclos entran en 15 minutos (900 segundos) divididos por la duración de un ciclo (216 segundos), el intérprete Lisp devolvió el número decimal `4.166666666666667`. Esto rompía la lógica del sistema, ya que requeríamos obtener exclusivamente la cantidad de **ciclos completos** (un número entero exacto, es decir, 4).

```
> (ciclos-por-tiempo 15 216)
4.166666666666667
```

Causa conceptual: La función de división primitiva `/` realiza cálculos exactos. En este entorno de ejecución, al dividir esos enteros, el resultado se representó como un

número de coma flotante para no perder los restos (indicando que entran 4 ciclos y un "pedazo" del quinto).

Solución aplicada: Se envolvió la operación aritmética completa con la primitiva ``floor ...``. Esta función procesa cualquier número real y lo trunca hacia el entero inferior más cercano. De esta manera, LISP procesa el ``4.1666...``, descarta los decimales (el ciclo incompleto) y devuelve el valor entero ``4``, manteniendo la pureza de la función y evitando errores de tipado. **(Se adjunta captura de pantalla en consola demostrando el retorno decimal original).**

Bug N° 2: Manejo incorrecto del tiempo Unix debido al desfase de Épocas

Durante el desarrollo del temporizador surgió un problema relacionado con el manejo de fechas y horas en Lisp. Inicialmente se asumió que la función estándar `get-universal-time` devolvía directamente un timestamp Unix, por lo que los valores obtenidos se utilizaron sin realizar ninguna conversión.

Problema detectado

Al comparar los valores obtenidos con herramientas externas de conversión de tiempo Unix, se observó que las fechas resultantes eran incorrectas y presentaban una diferencia significativa respecto del tiempo real.

Convierte epoch a una fecha legible y viceversa

Epoch a Fecha

Admite marcas de tiempo Unix en segundos, milisegundos y microsegundos.

Tiempo Unix en **segundos**:

UTC: viernes, 1 de junio de 2096, 21:51:41

Hora local: viernes, 1 de junio de 2096, 18:51:41 UTC-03:00

Tiempo relativo: dentro de 70 años

Causa del error

La causa del problema fue una interpretación incorrecta del funcionamiento de `get-universal-time`. Esta función devuelve la cantidad de segundos transcurridos desde el 1 de enero de 1900, mientras que el formato Unix utiliza como referencia el 1 de enero de 1970. Debido a esta diferencia de épocas, los timestamps obtenidos no eran compatibles con el estándar Unix.

Proceso de investigación

Para comprender el problema se consultó documentación de Lisp (<https://lisp-docs.github.io/>) y recursos externos como Wikipedia y Lisp Docs. Además, se verificaron los resultados utilizando un conversor de timestamps Unix para confirmar la diferencia observada.

Solución implementada

Se utilizó la función `encode-universal-time` para calcular el valor correspondiente al 1 de enero de 1970 en formato Universal Time. Posteriormente se restó dicho valor al resultado obtenido mediante `get-universal-time`, logrando así convertir correctamente el tiempo de Lisp a tiempo Unix.

Una vez aplicada esta corrección, los timestamps generados coincidieron con los valores esperados. Como resultado, se implementó la función auxiliar `obtener-timestamp`, encargada de realizar esta conversión de forma automática.

Convierte epoch a una fecha legible y viceversa

Epoch a Fecha

Admite marcas de tiempo Unix en segundos, milisegundos y microsegundos.

Tiempo Unix en **segundos**:

UTC: martes, 2 de junio de 2026, 19:08:49

Hora local: martes, 2 de junio de 2026, 16:08:49 UTC-03:00

Tiempo relativo: hace 3 horas

Bug N° 3: Desbordamiento de pila por recursividad mal planteada

Descripción del error:

La primera versión de la función auditoria generaba un error de *stack overflow* (desbordamiento de pila). La función se ejecutaba recursivamente de forma continua para verificar cambios en el estado de la luz, sin ningún mecanismo de espera o control de frecuencia de ejecución.

Causa:

En la rama donde no se detectaba un cambio de color, la función volvía a llamarse a sí misma inmediatamente.

Esto provocaba una cantidad excesiva de llamadas recursivas en muy poco tiempo, acumulando marcos en la pila hasta alcanzar el límite de memoria disponible.

Durante la ejecución se obtuvo el siguiente mensaje de error:

```
Break 8 [10]> (defun auditoria (color-inicio)
  (let ((color-nuevo (timer (obtener-timestamp))))
    (cond
      ((not (equal color-inicio color-nuevo))
        (format t "Tiempo ~a: La luz ha cambiado de ~a a ~a"
          (obtener-timestamp) color-inicio color-nuevo)
        )
      (t (auditoria color-nuevo))
    )
  )
)

AUDITORIA
Break 8 [10]> (auditoria (timer (obtener-timestamp)))

*** - Program stack overflow. RESET
```

Solución implementada:

Se incorporó una función sleep antes de realizar la nueva llamada recursiva, de manera que la auditoría se ejecute periódicamente y no de forma continua. Se estableció una pausa de **6 segundos**, ya que corresponde al tiempo mínimo observado para que el semáforo cambie de estado (por ejemplo, de amarillo a verde). Esto redujo significativamente la cantidad de llamadas recursivas y evitó el desbordamiento de la pila.

Resultado:

La función comenzó a monitorear correctamente los cambios de estado sin generar errores de memoria ni saturar la pila de ejecución.

Bug N° 4: Corrección Logica

Consigna:

La consigna solicita agregar un tiempo de intermitencia de 3 segundos entre los cambios de luces.

Problema detectado:

Durante la realización de la extensión ocurrió un error con la función de **validar-transiciones-p**, que no devolvió la lista con el nombre de "cambiar-a-<color>" especificado en la consigna;

Especificación:

- **Entrada:** `color-actual` (símbolo: `en-rojo`, `en-amarillo`, `en-verde`) y `cambiar-a` (símbolo del color destino: `rojo`, `amarillo`, `verde`)
- **Salida:** devuelve una lista con el estado y una acción a realizar, esta última como literal `"cambiar-a-<color>"`.
- **Comportamiento:** Por defecto, retorna color actual y `'accion-por-defecto` si la transición no es válida

Causa del error:

Durante la implementación de las intermitencias se modificaron los estados del semáforo, sin embargo, la función ***timer*** devolvía estados con nombres `rojo`, `amarillo` y `verde` mientras que ***validar-transiciones-p*** evaluaba estados con el formato `en-rojo`, `en-amarillo` y `en-verde`. Debido a esta diferencia de nomenclatura, las comparaciones realizadas retornaban `NIL`, provocando que transiciones válidas fueran rechazadas y que la función transición no generará la acción `"cambiar-a-<color>"`.

Solución al error:

Se eliminó el prefijo “en-” de las comparaciones para unificar la representación de los estados del semáforo en todo el sistema. Así los estados evaluados por dicha función coinciden con los estados retornados por `timer`, permitiendo que las transiciones válidas sean reconocidas correctamente y genere la acción correspondiente.

Si bien esta solución se aparta parcialmente de la nomenclatura presentada en el ejemplo de la consigna, se decidió mantener una única convención de nombres dentro del programa para garantizar la consistencia y evitar errores de comparación entre símbolos.

```
(defun validar-transiciones-p (color-actual cambiar-a)
  (cond
    ((and (equal color-actual 'en-rojo) (equal cambiar-a 'rojo-intermitente)) t)
    ((and (equal color-actual 'en-rojo-intermitente) (equal cambiar-a 'amarillo)) t)
    ((and (equal color-actual 'en-amarillo) (equal cambiar-a 'amarillo-intermitente)) t)
    ((and (equal color-actual 'en-amarillo-intermitente) (equal cambiar-a 'verde)) t)
    ((and (equal color-actual 'en-verde) (equal cambiar-a 'verde-intermitente)) t)
    ((and (equal color-actual 'en-verde-intermitente) (equal cambiar-a 'rojo)) t)
    ((and (equal color-actual 'rojo) (equal cambiar-a 'rojo-intermitente)) t)
    ((and (equal color-actual 'rojo-intermitente) (equal cambiar-a 'amarillo)) t)
    ((and (equal color-actual 'amarillo) (equal cambiar-a 'amarillo-intermitente)) t)
    ((and (equal color-actual 'amarillo-intermitente) (equal cambiar-a 'verde)) t)
    ((and (equal color-actual 'verde) (equal cambiar-a 'verde-intermitente)) t)
    ((and (equal color-actual 'verde-intermitente) (equal cambiar-a 'rojo)) t)
    (t nil)
  )
)
```

Corrección del código original:-En el requerimiento 2 ***validar-transiciones-p*** se añadieron las transiciones a los colores intermitentes

```

(defun validar-transiciones-p (color-actual cambiar-a)
  (cond
    ((and (equal color-actual 'verde) (equal cambiar-a 'verde-intermitente)) t)
    ((and (equal color-actual 'verde-intermitente) (equal cambiar-a 'rojo)) t)
    ((and (equal color-actual 'rojo) (equal cambiar-a 'rojo-intermitente)) t)
    ((and (equal color-actual 'rojo-intermitente) (equal cambiar-a 'amarillo)) t)
    ((and (equal color-actual 'amarillo) (equal cambiar-a 'amarillo-intermitente)) t)
    ((and (equal color-actual 'amarillo-intermitente) (equal cambiar-a 'verde)) t)
    (t nil)
  )
)

```

-En el requerimiento 3 **timer** se añadieron los intermitentes y se sumaron los 3 segundos de cada intermitente al timestamp. Las posiciones pasaron de **216 -> 225**

```

(defun timer (timestamp)
  (if (numberp timestamp)
    (let ((posicion (rem timestamp 225)))
      (cond
        ((< posicion 120) 'verde)
        ((< posicion 123) 'verde-intermitente)
        ((< posicion 213) 'rojo)
        ((< posicion 216) 'rojo-intermitente)
        ((< posicion 222) 'amarillo)
        (t 'amarillo-intermitente)))
    (format t "El timestamp debe ser un valor numérico")
  )
)

```

En el requerimiento 6 **distribucion-temporal** se añadieron los 3 intermitentes para que también calcule el porcentaje de tiempo de los colores intermitentes.

```

(defun distribucion-temporal (horas)
  (if (or (not (numberp horas)) (<= horas 0))
      'error parametro-invalido)

  (let* ((tiempo-total (* horas 3600))
         (ciclos (floor (/ tiempo-total 225)))
         (resto (mod tiempo-total 225))

         (verde (+ (* ciclos 120)
                   (min resto 120)))

         (verde-intermitente (+ (* ciclos 3)
                                 (max 0
                                     (min 3
                                           (- resto 120))))))

         (rojo (+ (* ciclos 90)
                  (max 0
                      (min 90
                          (- resto 123)))))

         (rojo-intermitente (+ (* ciclos 3)
                                (max 0
                                    (min 3
                                          (- resto 213)))))

         (amarillo (+ (* ciclos 6)
                      (max 0
                          (min 6
                              (- resto 216)))))

         (amarillo-intermitente (+ (* ciclos 3)
                                    (max 0
                                        (min 3
                                              (- resto 222)))))

        )
    )

```

```

(list
  (list 'verde
        (/ (round (* (/ verde tiempo-total) 10000))
           100.0))

  (list 'verde-intermitente
        (/ (round (* (/ verde-intermitente tiempo-total) 10000))
           100.0))

  (list 'rojo
        (/ (round (* (/ rojo tiempo-total) 10000))
           100.0))

  (list 'rojo-intermitente
        (/ (round (* (/ rojo-intermitente tiempo-total) 10000))
           100.0))

  (list 'amarillo
        (/ (round (* (/ amarillo tiempo-total) 10000))
           100.0))

  (list 'amarillo-intermitente
        (/ (round (* (/ amarillo-intermitente tiempo-total) 10000))
           100.0)))
)

```

Bug N° 5: Error de diseño: anidamiento innecesario de estructuras condicionales

Durante el proceso de migración de la función timer a Clojure, se identificó un problema de diseño en su implementación original en Lisp. La función utilizaba una estructura if para validar el tipo de dato recibido y, dentro de ella, una estructura cond para determinar el estado correspondiente del semáforo. Si bien esta implementación

era funcionalmente correcta, implicaba un anidamiento de estructuras condicionales que la cátedra considera una mala práctica de diseño.

La estructura `if` se empleaba para verificar que el parámetro `timestamp` fuera de tipo numérico antes de realizar cálculos con él. Esta validación es necesaria, ya que la función utiliza operaciones aritméticas que provocarían errores si se recibiera un valor de otro tipo.

```
OBTENER-TIMESTAMP
[2]> (defun timer (timestamp)
      (let ((posicion (rem timestamp 225)))
        (cond
         ((< posicion 90) 'rojo)
         ((< posicion 93) 'rojo-intermitente)
         ((< posicion 213) 'verde)
         ((< posicion 216) 'verde-intermitente)
         ((< posicion 222) 'amarillo)
         (t 'amarillo-intermitente))
        )
      )
)

TIMER
[3]> (timer (obtener-timestamp))

ROJO
[4]> (timer "Esto va a fallar")

*** - REM: "Esto va a fallar" is not a real number
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead.
ABORT          :R2      Abort main loop
Break 1 [5]>
```

Error al llamar a la función sin validar datos.

Para mejorar el diseño de la función y evitar el anidamiento de condicionales, se reemplazó dicha lógica por una estructura `typecase`. De esta manera, el cálculo de la posición dentro del ciclo del semáforo solo se ejecuta cuando el valor recibido es numérico. En caso contrario, la función devuelve un mensaje informando que el dato ingresado no es válido.

Esta modificación permite mantener la validación de tipos requerida, mejorar la legibilidad del código y adecuar la implementación a los criterios de diseño establecidos por la cátedra.

```

Break 1 [5]> (defun timer (timestamp)
  (typecase timestamp
    (number
      (let ((posicion (rem timestamp 225)))
        (cond
          ((< posicion 90) 'rojo)
          ((< posicion 93) 'rojo-intermitente)
          ((< posicion 213) 'verde)
          ((< posicion 216) 'verde-intermitente)
          ((< posicion 222) 'amarillo)
          (t 'amarillo-intermitente)) ))
      (t (format t "El timestamp debe ser un valor numérico"))))
  )
)

TIMER
Break 1 [5]> (timer (obtener-timestamp))

ROJO
Break 1 [5]> (timer "Esto va a fallar")
El timestamp debe ser un valor numérico
NIL
Break 1 [5]>

```

Funcion llamada con validacion.

Requerimiento 7: Aseguramiento de la Calidad

A continuación, se presentan los escenarios de prueba diseñados para validar el correcto funcionamiento de cada uno de los requerimientos del sistema. Los siguientes bloques de código están listos para ser copiados y ejecutados en la consola del entorno de desarrollo (SBCL):

Requerimiento 1: Función transicion (y validación)

Funcionamiento normal: Evalúa una transición estándar y lineal del semáforo.

(transicion 'rojo 'rojo-intermitente)

Camino alternativo: Evalúa la transición de recuperación del flujo hacia el estado verde.

(transicion 'rojo-intermitente 'verde)

Caso de error: Intento de cambio directo inválido que fuerza el retorno de la acción por defecto. **(transicion 'rojo 'verde)**

Requerimiento 2: Función timer (Temporizador Automático)

Funcionamiento normal: Evaluación del temporizador en una fase del ciclo (Rojo).

(timer 50)

Camino alternativo: Evaluación del temporizador en una fase avanzada del ciclo completo de 225s (Verde). **(timer 150)**

Caso de error: Inyección de un tipo de dato inválido (símbolo en lugar de un valor numérico entero). (**timer 'hola**)

Requerimiento 3: Función auditoria (Sistema de Logging)

Funcionamiento normal: Monitoreo y registro dinámico de los cambios en la terminal basándose en el reloj del sistema. (**auditoria (timer (obtener-timestamp)) 6 nil**)

Camino alternativo: Evaluación del comportamiento del monitor ante un estado estático forzado para auditar su respuesta. (**auditoria 'verde 0 nil**)

caso de error: Introducir un valor no valido en ciclos por ejemplo lanzara un mensaje de error: (auditoria 'verde 'seis nil)

Requerimiento 4: Análisis de Ciclos Semafóricos

Funcionamiento normal (4a. duracion-ciclo): Cálculo de la duración total sumando la lista de tiempos de las luces. (**duracion-ciclo '(90 3 120 3 6 3)**)

Funcionamiento normal (4b. recomendacion-ciclo): Evaluación del mensaje del sistema ante un ciclo extendido. (**recomendacion-ciclo 225**)

Camino alternativo (Recomendación óptima): Verificación del mensaje ante un ciclo estándar que entra en el rango óptimo. (**recomendacion-ciclo 90**)

Requerimiento 5: Planificación Temporal (Función ciclos-por-tiempo)

Funcionamiento normal: Cálculo de la cantidad de ciclos completos de 225 segundos que se completan en una hora (60 minutos). (**ciclos-por-tiempo 60 225**)

Camino alternativo: Verificación del comportamiento del cálculo con un tiempo menor a un ciclo completo (por ejemplo, 2 minutos). (**ciclos-por-tiempo 2 225**)

Caso de error: Intento de división por cero o paso de parámetros nulos al simulador. (**ciclos-por-tiempo 15 0**)

Requerimiento 6: Función distribucion-temporal

Funcionamiento normal: Cálculo de los porcentajes de tiempo asignados a cada fase del semáforo para un ciclo base. (**distribucion-temporal 1**)

Camino alternativo: Cálculo de la distribución estadística acumulada para un rango de tiempo con segundos restantes. (**distribucion-temporal 1.015**)

Camino alternativo: Cálculo de la distribución estadística acumulada para un rango corto de tiempo. (**distribucion-temporal 0.05**)

Caso de error: Falla controlada al ingresar un argumento simbólico no numérico que impide el cálculo matemático. (**distribucion-temporal 'hola'**)

Definición y Características de Clojure

Según clojure.org; Clojure es un lenguaje de programación dinámico y de propósito general que combina la accesibilidad y el desarrollo interactivo de un lenguaje de scripting con una infraestructura eficiente y robusta para la programación multihilo. Clojure es un lenguaje compilado, pero conserva su dinamismo: todas las características que admite están disponibles en tiempo de ejecución. Clojure facilita el acceso a los frameworks de Java, con sugerencias de tipo e inferencia de tipos opcionales, para garantizar que las llamadas a Java eviten la reflexión.

Clojure es un dialecto de Lisp y comparte con la filosofía de considerar el código como datos, así como un potente sistema de macros. Clojure es principalmente un lenguaje de programación funcional y cuenta con un amplio conjunto de estructuras de datos inmutables y persistentes. Cuando se requiere un estado mutable, Clojure ofrece un sistema de memoria transaccional de software y un sistema de agentes reactivos que garantizan diseños multihilo limpios y correctos.

Además de ser un lenguaje funcional moderno, Clojure fue diseñado por Rich Hickey con el objetivo de simplificar el desarrollo de aplicaciones concurrentes. Hickey identificó que muchos de los problemas más difíciles de los sistemas modernos surgen de la combinación de estado mutable compartido y ejecución concurrente. Por esta razón, Clojure adopta la inmutabilidad como principio fundamental y proporciona mecanismos específicos para gestionar cambios de estado de manera segura.

A diferencia de otros lenguajes que basan la concurrencia en bloqueos explícitos (locks) y sincronización manual, Clojure ofrece abstracciones de alto nivel como Atoms, Refs, Agents y Software Transactional Memory (STM), permitiendo desarrollar aplicaciones multihilo más seguras y fáciles de mantener.

Otra característica destacada es su ejecución sobre la Máquina Virtual de Java (JVM). Gracias a ello, Clojure hereda la robustez, portabilidad, optimización y rendimiento de la plataforma Java, incluyendo el recolector de basura, la compilación Just-In-Time (JIT) y el acceso a miles de bibliotecas existentes. Esta integración permite utilizar directamente clases y frameworks de Java desde código Clojure, reduciendo significativamente el esfuerzo necesario para implementar funcionalidades comunes

como acceso a bases de datos, manejo de fechas y horas, comunicación en red o procesamiento de archivos.

Áreas de aplicación de Clojure

Clojure se utiliza principalmente en el desarrollo de aplicaciones backend, sistemas distribuidos, procesamiento de datos, plataformas financieras, automatización y algunas áreas de inteligencia artificial. Su enfoque funcional, la inmutabilidad de los datos y su integración con la Máquina Virtual de Java (JVM) lo convierten en una opción adecuada para aplicaciones escalables y concurrentes.

1. Desarrollo Backend y APIs Web

Se utiliza para construir servidores, APIs REST y aplicaciones web de alto rendimiento. Entre sus aplicaciones más comunes se encuentran los sistemas bancarios, las plataformas de comercio electrónico y diversos servicios web empresariales.

2. Sistemas Distribuidos

Gracias a su modelo de inmutabilidad y concurrencia, resulta adecuado para aplicaciones que operan sobre múltiples servidores o nodos. Algunos ejemplos son los sistemas de microservicios, plataformas en la nube y sistemas de mensajería.

3. Procesamiento y Análisis de Datos

Puede emplearse para manipular grandes volúmenes de datos de forma eficiente. Entre sus usos se encuentran los procesos ETL (Extracción, Transformación y Carga de datos), el análisis estadístico y el procesamiento de eventos en tiempo real.

4. Tecnología Financiera (FinTech)

Es utilizado en aplicaciones financieras por su capacidad para gestionar operaciones concurrentes de manera segura. Algunos ejemplos son los sistemas de banca digital, plataformas de pago y herramientas de gestión de inversiones.

5. Inteligencia Artificial y Ciencia de Datos

Puede utilizarse para el desarrollo de herramientas de análisis de datos, procesamiento de información y aplicaciones relacionadas con aprendizaje automático, aprovechando las bibliotecas disponibles en el ecosistema de la JVM.

6. Automatización y Herramientas de Desarrollo

También se emplea para crear scripts, herramientas internas y procesos de automatización dentro de organizaciones y entornos empresariales.

Grandes empresas que utilizan Clojure

Según la información publicada por Clojure.org, existen aproximadamente 450 organizaciones que utilizan Clojure en entornos productivos. Entre los casos más destacados se encuentran:

- **Walmart**, que ha utilizado Clojure para desarrollar sistemas de gestión de datos capaces de dar soporte a más de 5000 tiendas e integrar información proveniente de canales físicos, web y móviles.
- **Atlassian**, que emplea Clojure para implementar funcionalidades de colaboración en tiempo real tanto en productos nuevos como en aplicaciones ya existentes.
- **The French Digital Administration**, organismo encargado de proveer infraestructura digital para organismos públicos de Francia. Dentro de esta organización, **Etalab**, el departamento responsable de la gestión de datos públicos, utiliza Clojure para la publicación y administración de los códigos fuente públicos del Estado francés.

Fase 3: Diferencias entre funciones de Lisp y Clojure

Luego de corregir el error identificado en la fase anterior, se procedió a migrar la función timer a Clojure para completar el requerimiento correspondiente a la Fase 3.

Durante este proceso se observó que Clojure no dispone de una estructura equivalente a typecase, utilizada en la implementación original en Lisp para validar el tipo de dato recibido. Debido a esta diferencia entre ambos lenguajes, fue necesario realizar un rediseño de la solución para obtener una traducción adecuada de la función.

Para resolver este inconveniente, se decidió separar la lógica de validación de tipos de la lógica encargada de calcular la posición dentro del ciclo del semáforo. Con este objetivo, se implementó una función auxiliar responsable del cálculo de la posición, la cual es invocada únicamente después de verificar que el valor recibido sea de tipo numérico.

Esta modificación permitió conservar el comportamiento original de la función, garantizar su correcto funcionamiento en Clojure y mejorar la organización del código mediante una separación más clara de responsabilidades.

```

(defn estados-semaforo [posicion]
  (cond
    (< posicion 90) :rojo
    (< posicion 93) :rojo-intermitente
    (< posicion 213) :verde
    (< posicion 216) :verde-intermitente
    (< posicion 222) :amarillo
    :else :amarillo-intermitente)
  )

;; =====
;; FUNCIÓN: timer
;; NATURALEZA: Pura (Dado un timestamp, siempre retorna el mismo color)
;; ESTRATEGIA: Selecccion simple
;; IMPACTO: No destructiva
;; =====
(defn timer [timestamp]
  (cond
    (not (number? timestamp)) (println "El timestamp debe ser un valor numerico")
    :else (estados-semaforo (mod timestamp 225))
  )
)

```

Funcion auxiliar implementada en clojure.

Sobre Clojure:

- 1. Ambos lenguajes son Lisps, pero Clojure elimina los símbolos cotizados ('rojo) en favor de los *Keywords* (:rojo). ¿Cuál es la ventaja técnica y de rendimiento de un Keyword en Clojure para mapear estados?

En Common Lisp los estados del semáforo pueden representarse mediante símbolos como 'rojo, 'verde o 'amarillo. Clojure utiliza keywords (:rojo, :verde, :amarillo), que son identificadores inmutables y autoevaluables diseñados especialmente para representar atributos, estados y claves de mapas. A diferencia de los símbolos, los keywords están "cached and interned" globalmente, por lo que una misma palabra clave se reutiliza en toda la ejecución del programa. Esto reduce el consumo de memoria y permite realizar comparaciones y búsquedas de manera muy eficiente. Además, los keywords se integran naturalmente con los mapas de Clojure, que son una de las estructuras de datos más utilizadas del lenguaje. En el simulador semafórico, representar los estados mediante :rojo, :verde y :amarillo facilita su almacenamiento y consulta en mapas, manteniendo una representación simple, segura y eficiente.

Sin embargo, en este caso la diferencia práctica respecto de Common Lisp no es tan grande, ya que los símbolos 'rojo, 'verde y 'amarillo también permiten modelar correctamente los estados del semáforo. La principal ventaja de los keywords aparece

cuando los estados se utilizan intensivamente como claves de mapas o estructuras de datos asociativas, que constituyen una parte central del estilo de programación en Clojure.

- 2. Clojure prohíbe la mutabilidad por defecto usando estructuras de datos persistentes. ¿Cómo maneja Clojure el paso del tiempo en el simulador semafórico sin modificar variables?

Clojure maneja el paso del tiempo mediante programación funcional y estructuras de datos persistentes. En lugar de modificar una variable que almacena el estado actual del semáforo, cada cálculo produce un nuevo valor a partir del tiempo recibido como entrada. Por ejemplo, la función obtiene el timestamp actual, calcula la posición dentro del ciclo semafórico mediante una operación módulo y devuelve el estado correspondiente. De esta manera no existe una variable mutable que cambie de valor, cada ejecución genera un nuevo resultado a partir de los datos de entrada.

Sin embargo, esta característica no es exclusiva de Clojure. En nuestra implementación en Common Lisp también se adopta un enfoque funcional, ya que el estado del semáforo se calcula directamente a partir del timestamp recibido, sin modificar variables ni mantener un estado global mutable. La diferencia es que Clojure refuerza este estilo mediante la inmutabilidad por defecto y el uso de estructuras de datos persistentes, mientras que Common Lisp permite combinar tanto paradigmas funcionales como imperativos.

Bibliografía

1. **Clojure.org (2026).** *Clojure Documentation, Learn Clojure and FAQ Guide.* Disponible en: <https://clojure.org> (Consultado para el análisis de características, la guía de aprendizaje, el ecosistema industrial y las preguntas frecuentes de la Fase 3).
2. **Clojure Cheatsheet (2026).** *Clojure Cheatsheet.* Disponible en: [Clojurie Cheatsheet](#) (Utilizado como referencia rápida de sintaxis para el desarrollo en Clojure de la Fase 3).
3. **Clojure Online Compiler (2026).** *Online Compiler.* Disponible en: [Clojure online compiler](#) (Utilizado como entorno de pruebas interactivo para el desarrollo en Clojure de la Fase 3).
4. **Local-Time Docs (2026).** *Common-lisp-libraries.* Disponible en: [Local-time docs](#) (Fuente oficial utilizada para la documentación, justificación e implementación del módulo de gestión de timestamps Unix en la Fase 2).

5. **Quicklisp Beta (2026).** *Quicklisp Library Manager for Common Lisp*. Disponible en: <https://www.quicklisp.org/beta/> (Consultado para la documentación de instalación y carga de la biblioteca local-time en la Fase 2).